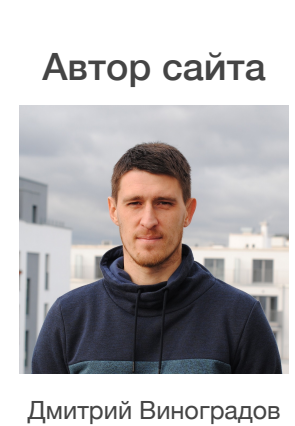
Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Новогодняя скидка 50%
на набор курсов:
Jetpack Compose
Kotlin Coroutines
Dagger + Hilt
Testing

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid
15 086 подписчиков
Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 34. Анимация. AnimatedVisibility, AnimatedContent

28 ноября 2023

В этом уроке рассматриваем оставшиеся верхнеуровневые инструменты анимации: AnimatedVisibility, AnimatedContent, Crossfade, animate, animateContentSize

AnimatedVisibility

AnimatedVisibility позволяет анимированно показать/скрыть контент.

Пример:

```
@Composable
fun HomeScreen() {
   var checked by remember { mutableStateOf(true) }
   Column {
       Checkbox(checked = checked, onCheckedChange = {checked = it})
       AnimatedVisibility(visible = checked) {
           Text("Some description", fontSize = 30.sp)
       }
   }
}
```

Результат:

Текст выезжает и заезжает обратно.

Тот же самый код, но без анимации:

```
@Composable
fun HomeScreen() {
   var checked by remember { mutableStateOf(true) }
   Column {
       Checkbox(checked = checked, onCheckedChange = { checked = it })
       if (checked)
           Text("Some description", fontSize = 30.sp)
   }
}
```

Выглядит уже не так красиво:

Анимации

Мы можем сами определять, какая анимация будет использована. Нам доступны 4 типа анимации: Fade, Expand/Shrink, Scale и Slide

По умолчанию используется примерно такой набор анимаций:

```
AnimatedVisibility(
   visible = checked,
   enter = fadeIn() + expandVertically(),
   exit = fadeOut() + shrinkVertically()
) {
   Text("Some description", fontSize = 30.sp)
}
```

enter - анимация появления
exit - анимация исчезания

У этих анимаций есть свои параметры. Например, можно задать спецификацию. Используем tween с продолжительностью в 2 секунды:

```
private const val DRTN = 2000

@Composable
fun HomeScreen() {
   var checked by remember { mutableStateOf(true) }
   Column {
       Checkbox(checked = checked, onCheckedChange = {checked = it})
       AnimatedVisibility(
           visible = checked,
           enter = fadeIn(tween(DRTN)) + expandVertically(tween(DRTN)),
           exit = fadeOut(tween(DRTN)) + shrinkVertically(tween(DRTN))
       ) {
           Text("Some description", fontSize = 30.sp)
       }
   }
}
```

Результат:

animateEnterExit

Если внутри AnimatedVisibility находятся несколько UI элементов, то каждому из них можно присвоить отдельную анимацию. Для этого используется Modifier.animateEnterExit.

Пример:

```
private const val DRTN = 1000


@OptIn(ExperimentalAnimationApi::class)
@Composable
fun HomeScreen() {
   var checked by remember { mutableStateOf(true) }
   Column {
       Checkbox(checked = checked, onCheckedChange = {checked = it})
       AnimatedVisibility(
           visible = checked,
           enter = fadeIn(tween(DRTN)),
           exit = fadeOut(tween(DRTN))
       ) {
           Row() {
               Text(
                   "1", fontSize = 30.sp, modifier = Modifier.animateEnterExit(
                       enter = slideInHorizontally(tween(DRTN)) { -it },
                       exit = slideOutHorizontally(tween(DRTN)) { -it }
                   )
               )
               Text(
                   "2", fontSize = 30.sp, modifier = Modifier.animateEnterExit(
                       enter = slideInVertically(tween(DRTN)) { -it },
                       exit = slideOutVertically(tween(DRTN)) { -it }
                   )
               )
               Text(
                   "3", fontSize = 30.sp, modifier = Modifier.animateEnterExit(
                       enter = slideInVertically(tween(DRTN)) { it },
                       exit = slideOutVertically(tween(DRTN)) { it }
                   )
               )
               Text(
                   "4", fontSize = 30.sp, modifier = Modifier.animateEnterExit(
                       enter = slideInHorizontally(tween(DRTN)) { it },
                       exit = slideOutHorizontally(tween(DRTN)) { it }
                   )
               )
           }
       }
   }
}
```

У нас тут строка с 4 текстами. Без анимации выглядит, как обычный текст - 1234. Но для каждой цифры настроена своя анимация, и цифры разъезжаются в разные стороны.

Результат:

Учитывайте, что анимация из animateEnterExit складывается с анимацией, которая указана в параметрах enter/exit функции AnimatedVisibility. Даже если вы явно не указывали анимацию в AnimatedVisibility, там есть анимация по умолчанию. Поэтому если вам надо полностью ее отключить, используйте None:

```
AnimatedVisibility(
    visible = checked,
    enter = EnterTransition.None,
    exit = ExitTransition.None
)
```

AnimatedVisibility -> transition

AnimatedVisibility для своей работы под капотом создает transition. У этого transition вызываются различные animate* методы (в соответствии с анимациями, которые мы указали в enter/exit), и полученные State собираются в Modifier, который в итоге и отвечает за анимацию контента:

```
modifier.graphicsLayer {
   this.alpha = alpha
   this.scaleX = scale
   this.scaleY = scale
   this.transformOrigin = transformOrigin
}
```

Мы тоже можем воспользоваться этим transition для своих целей.

Например, добавим изменения цвета при скрытии/показе:

```
private const val DRTN = 2000


@OptIn(ExperimentalAnimationApi::class)
@Composable
fun HomeScreen() {
   var checked by remember { mutableStateOf(true) }
   Column {
       Checkbox(checked = checked, onCheckedChange = {checked = it})
       AnimatedVisibility(
           visible = checked,
           enter = expandVertically(tween(DRTN)),
           exit = shrinkVertically(tween(DRTN))
       ) {
           val color by transition.animateColor(
               transitionSpec = { tween(DRTN) },
               label = "color"
           ) { state ->
               if (state == EnterExitState.Visible) Color.Green else Color.Red
           }
           Text("Some description", fontSize = 30.sp, color = color)
       }
   }
}
```

Вызываем transition.animateColor, чтобы получить State для анимации цвета. Если контент становится видимым, цвет будет меняться на зеленый. При закрытии - на красный.

Результат:

00:00 00:00

transition - > AnimatedVisibility

Связка AnimatedVisibility и transition работает и в обратную сторону. Если у нас есть transition, мы можем привязать к нему AnimatedVisibility. В этом случае AnimatedVisibility будет использовать этот transition вместо создания своего.

Для демонстрации используем пример с прошлого урока про transition. Добавим в него AnimatedVisibility:

```
@OptIn(ExperimentalAnimationApi::class, ExperimentalTransitionApi::class)
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = hiltViewModel()
) {
   Column {
       val transition = updateTransition(targetState = homeViewModel.visitorCount.collectAsState().
       val animatedCount = transition.animateInt(transitionSpec = { tween(1000) }, label = "Count")
           state
       }
       val animatedWidth = transition.animateDp(transitionSpec = { tween(1000) }, label = "Width")
           (state / 500F * 300F).dp
       }

       val alertTransition = transition.createChildTransition { state ->
           state > 300
       }
       val animatedColor = alertTransition.animateColor(transitionSpec = { tween(1000) }, label = "
           if (state) Color.Red else Color.Green
       }

       Text(
           text = "Visitors: ${animatedCount.value}",
           fontSize = 30.sp
       )
       Spacer(
           modifier = Modifier
               .height(40.dp)
               .width(animatedWidth.value)
               .background(animatedColor.value)
       )
       alertTransition.AnimatedVisibility(
           visible = { state -> state },
           enter = expandHorizontally(tween(1000)),
           exit = shrinkHorizontally(tween(1000))
       ) {
           Text("Alert", fontSize = 30.sp)
       }
   }
}
```

Идем по порядку.

Создаем transition, который стартует дочерние анимации при изменении количества посетителей в homeViewModel.visitorCount

Создаем дочерние анимации animatedCount и animatedWidth для анимации текста и ширины шкалы.

Создаем дочерний alertTransition чтобы дальше работать не с количеством посетителей, а с Boolean состоянием тревоги.

От alertTransition создаем animatedColor для анимации цвета. Если Alert - то красный, иначе - зеленый.

И в конце используем alertTransition.AnimatedVisibility, чтобы дополнительно показывать/скрывать текст Alert.

Результат:

Т.е. получается такая иерархия.

transition (число посетителей)
  animatedCount (число посетителей)
  animatedWidth (число посетителей -> ширина)
  alertTransition (число посетителей -> тревога)
    animatedColor (тревога -> цвет)
    AnimatedVisibility (тревога -> отображение текста)

Корневой transition получает число посетителей, раскидывает его по своим дочерним элементам, а те уже преобразуют его в свои target-значения.

AnimatedContent

Дает возможность анимированно переключаться между контентом.

Простой пример:

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = hiltViewModel()
) {
   AnimatedContent(
       targetState = homeViewModel.visitorCount.collectAsState().value,
       label = "Visitors"
   ) { count ->
       Text("$count", fontSize = 30.sp)
   }
}
```

Результат:

Старое значение исчезает, новое появляется

Без анимации, это выглядело бы так:

```
@OptIn(ExperimentalAnimationApi::class)
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = hiltViewModel()
) {
   Text("${homeViewModel.visitorCount.collectAsState().value}", fontSize = 30.sp)
}
```

Результат:

В transitionSpec мы можем указать анимации появления нового значения и скрытия старого. По умолчанию там идет примерно такой набор, только с кучей всяких параметров:

```
AnimatedContent(
   targetState = homeViewModel.visitorCount.collectAsState().value,
   label = "Visitors",
   transitionSpec = {
       fadeIn() + scaleIn() with fadeOut()
   },
   content = { count ->
       Text("$count", fontSize = 30.sp)
   }
)
```

fadeIn() + scaleIn() для нового контента, fadeOut - для старого

Поменяем их на slide:

```
@OptIn(ExperimentalAnimationApi::class)
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = hiltViewModel()
) {
   AnimatedContent(
       targetState = homeViewModel.visitorCount.collectAsState().value,
       label = "Visitors",
       transitionSpec = {
           slideInVertically { it } with slideOutVertically { -it }
       },
       content = { count ->
           Text("$count", fontSize = 30.sp)
       }
   )
}
```

Результат:

Теперь новое значение выезжает снизу и вытесняет старое

Анимация может зависеть от значения. Например, можно сделать так, чтобы новое значение выезжало снизу, если оно больше старого, и сверху, если оно меньше старого. Для этого в transitionSpec нам доступны initialState (старое значение) и targetState (новое значение).

Делаем проверку какое значение больше, и указываем нужную slide анимацию:

```
AnimatedContent(
   targetState = homeViewModel.visitorCount.collectAsState().value,
   label = "Visitors",
   transitionSpec = {
       if (initialState < targetState) {
           slideInVertically { it } with slideOutVertically { -it }
       } else {
           slideInVertically { -it } with slideOutVertically { it }
       }
   },
   content = { count ->
       Text("$count", fontSize = 30.sp)
   }
)
```

Результат:

Под капотом у AnimatedContent работает transition. Когда происходит смена значения, запускается один AnimatedVisibility для скрытия старого контента и один AnimatedVisibility для отображения нового.

Crossfade

Если при смене контента нужна только fade-анимация, то можно использовать Crossfade

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = hiltViewModel()
) {
   Crossfade(
       targetState = homeViewModel.visitorCount.collectAsState().value,
       animationSpec = tween(1000),
       label = "Visitors"
   ) { visitors ->
       Text("$visitors", fontSize = 30.sp)
   }
}
```

Результат:

Под капотом тоже transition, который дает State для анимации прозрачности контента.

animate

Функция animate - это простой способ стартовать анимацию не в Composable коде. Т.е., например, в LaunchedEffect, onClick или ViewModel.

```
@Composable
fun HomeScreen() {
   val scope = rememberCoroutineScope()
   var counter by remember { mutableStateOf(0) }
   Text(text = "Count $counter",fontSize = 30.sp, modifier = Modifier.clickable {
       scope.launch {
           animate(
               typeConverter = Int.VectorConverter,
               initialValue = counter,
               targetValue = counter + 100,
               animationSpec = tween(1000)
           ) { value, _ ->
               counter = value
           }
       }
   })
}
```

Под капотом создается TargetBasedAnimation, который в цикле рассчитывает значение и отправляет его в блок кода.

Результат:

Modifier.animateContentSize

animateContentSize позволяет задать спецификацию для анимации изменения размера

Простой пример, в котором периодически меняется длина текста:

```
@Composable
fun HomeScreen(
   homeViewModel: HomeViewModel = hiltViewModel()
) {
   Box(Modifier.padding(16.dp)) {
       Text(
           text = "${homeViewModel.visitorCount.collectAsState().value}",
           fontSize = 30.sp,
           modifier = Modifier
               .border(width = 1.dp, color = Color.Black, shape = RoundedCornerShape(8.dp))
               .padding(8.dp)
               .animateContentSize(tween(1000))
       )
   }
}
```

Результат:

Performance

Когда мы анимируем какой-либо Modifier, то по возможности лучше делать это в lambda версии, чем в обычной. Самый наглядный пример - это offset
Например у нас есть некий offsetState, в котором анимируется значение. И мы используем его в Modifier.offset

```
modifier = Modifier.offset(x = offsetState.value)
```

Это приведет к избыточной рекомпозиции окружающего кода. Каждое новое значение будет вызывать рекомпозицию.

Поэтому этот вариант более эффективный:

```
modifier = Modifier
   .offset {
       IntOffset(offsetState.value, 0)
   }
```

Он позволит избежать рекомпозиций.

Аналогично вместо Modifier.alpha, Modifier.scale, Modifier.rotate используйте Modifier.graphicsLayer, который работает через лямбду.

Выбор

За 6 уроков мы рассмотрели практически все основные механизмы, в том числе и низкоуровневые. Остается только одна проблема - решить, какой именно из всех этих механизмов использовать.

Официальная документация дает целую схему, которая поможет сделать правильный выбор.

Присоединяйтесь к нам в **Telegram**:

- в канале StartAndroid публикуются ссылки на новые статьи с сайта startandroid.ru и интересные материалы с хабра, medium.com и т.п.

- в чатах решаем возникающие вопросы и проблемы по различным темам: Android, Compose, Kotlin, RxJava, Dagger, Тестирование, Performance

- ну и если просто хочется поговорить с коллегами по разработке, то есть чат Флудильня

Назад

Комментарии

# **Видео не отображаются** — **Илья** 06.02.2024 23:29
В Файерфоксе у меня видео не отображаются.
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.